

Score Breakdown (10-Point Scale)**Section I (Multiple-Choice):** 12 items $\times$ 0.5 pts = **6.0 pts** (60% Easy/Basic)**Section II (Applied Coding):** 3 problems = **4.0 pts** (40% Medium)**Total Exam Score: 10.0 points (100%)****NATIONAL ECONOMICS UNIVERSITY
FACULTY OF DATA SCIENCE & AI****OFFICIAL SOLUTIONS & GRADING GUIDE****Course:** Data Analysis with Python (DSAI1005)**Content:** NumPy (20%), Pandas (20%), Viz (20%), Code (40%)**Difficulty:** 60% Easy/Basic + 40% Medium — **Code:**
301-EN**SECTION I: MULTIPLE-CHOICE KEY & RATIONALE (6.0 pts — 0.5 pt each)***(60% Easy & Basic Questions: Foundational Knowledge & Core Syntax Verification)*

Question	1	2	3	4	5	6	7	8	9	10	11	12
Topic	NumPy (20%)				Pandas (20%)				Visualization (20%)			
Key	B	A	C	B	D	A	C	A	B	D	C	A

Technical Rationales for Multiple-Choice Items

- **Q1 (Key B):** `np.arange(start=2, stop=11, step=2)` generates sequence values starting at 2 up to, but excluding, 11 with increment 2: `[2, 4, 6, 8, 10]`.
- **Q2 (Key A):** The nested list `[[10, 20, 30], [40, 50, 60]]` has 2 rows and 3 columns. Thus, its shape is `(2, 3)` and its number of dimensions (`ndim`) is 2.
- **Q3 (Key C):** In standard NumPy 2D array indexing `m[row, col]`, row index 1 and column index 2 accesses the element with value 6.
- **Q4 (Key B):** In NumPy 2D operations, `axis=0` specifies aggregation along rows (down each column). For shape `(4, 5)`, computing the mean along `axis=0` produces a 1D array of 5 column means.
- **Q5 (Key D):** A Pandas `Series` is a 1-dimensional labeled array supporting homogeneous types, while a `DataFrame` is a 2-dimensional labeled tabular data structure with columns potentially of different types.
- **Q6 (Key A):** `df.iloc` is strictly integer position-based (0 to `length-1`). `df.iloc[0]` retrieves the first row, whereas `df.loc` indexes by label names.
- **Q7 (Key C):** `df.isna()` (or `df.isnull()`) produces a boolean `DataFrame` of the same shape, and calling `.sum()` sums the boolean flags (`True=1`) along columns to report the total missing count per column.
- **Q8 (Key A):** Pandas boolean filtering requires bitwise operators (`&` for AND, `|` for OR) and explicit parentheses around each condition: `df[(df['City'] == 'Hanoi') & (df['Age'] >= 20)]`. The Python logical operator `and` causes an ambiguity error on `Series`.
- **Q9 (Key B):** A line chart is standard for time-series data because the connecting lines clearly communicate continuous trajectories, temporal trends, and cyclical patterns across chronological order.

- **Q10 (Key D):** A box plot (`sns.boxplot`) provides a visual summary of the 5-number summary (min, Q_1 , median, Q_3 , max) and plots individual markers for outliers falling outside $1.5 \times IQR$ fences.
- **Q11 (Key C):** The `hue` parameter maps categorical levels to distinct color palettes, enabling visual grouping and segmentation without creating separate subplots.
- **Q12 (Key A):** `plt.tight_layout()` automatically adjusts subplot parameters and margins so that titles, tick marks, and axis labels fit within the figure without overlapping.

SECTION II: APPLIED CODING SOLUTIONS & RUBRIC (4.0 pts)

(40% Medium-Level Integrated Problems: Vectorized Pipelines, Tabular Analysis & Visual Analytics)

Question 1: Retail Sales Analysis with NumPy & Pandas (2.0 pts)

Standard Model Solution

```
# a) Vectorized Revenue calculation (0.4 pts):
df['Revenue'] = df['Price'] * df['Quantity'] * (1 - df['Discount'])

# b) Conditional Priority classification with np.where (0.4 pts):
df['Priority'] = np.where((df['Revenue'] >= 1000) | (df['Quantity'] >= 5), 'High', 'Standard')

# c) Filter Tech orders with Quantity >= 2 (0.4 pts):
tech_orders = df[(df['Category'] == 'Tech') & (df['Quantity'] >= 2)]

# d) Groupby Category and compute sum and mean (0.4 pts):
summary = df.groupby('Category')['Revenue'].agg(['sum', 'mean'])
# Alternative acceptable syntax:
# summary = df.groupby('Category').agg({'Revenue': ['sum', 'mean']})

# e) Sort summary table by total revenue descending (0.4 pts):
final_sorted = summary.sort_values(by='sum', ascending=False)
# If multi-level column indexing is produced: summary.sort_values(by=('Revenue', 'sum'), ascending=False)
```

Grading Criteria & Partial Credit for Question 1

- **Part a (0.4 pts):**

- 0.2 pts for correct column arithmetic (`df['Price'] * df['Quantity']`).
- 0.2 pts for discount application: `* (1 - df['Discount'])`. Equivalent arithmetic is awarded full credit.

- **Part b (0.4 pts):**

- 0.2 pts for compound condition with parentheses and bitwise OR: `(df['Revenue'] >= 1000) | (df['Quantity'] >= 5)`.
- 0.2 pts for correct assignment of output labels 'High' and 'Standard'.

- **Part c (0.4 pts):**

- 0.2 pts for filtering on Category: `df['Category'] == 'Tech'`.
- 0.2 pts for combining with & and `df['Quantity'] >= 2`. Deduct 0.1 pt if parentheses are missing.

- **Part d (0.4 pts):**

- 0.2 pts for grouping by 'Category'.
- 0.2 pts for aggregating 'Revenue' with `['sum', 'mean']`.

- **Part e (0.4 pts):**

- 0.2 pts for identifying sort column 'sum' (or `('Revenue', 'sum')`).
- 0.2 pts for `ascending=False`.

Question 2: Sales Performance Visualization with Matplotlib & Seaborn (1.5 pts)**Standard Model Solution**

```
# a) Matplotlib OO Canvas Initialization (0.5 pts):  
fig, axes = plt.subplots(1, 2, figsize=(12, 4))  
  
# b) Plotting on subplots (1.0 pts):  
# Subplot 1: Bar chart of average Revenue on axes[0] (0.5 pts):  
sns.barplot(data=df, x='Category', y='Revenue', ax=axes[0])  
  
# Subplot 2: Box plot of Revenue on axes[1] (0.5 pts):  
sns.boxplot(data=df, x='Category', y='Revenue', ax=axes[1])
```

Grading Criteria & Partial Credit for Question 2

- **Part a (0.5 pts):** Award 0.25 pts for subplot grid configuration (1, 2 for 1 row, 2 columns); award 0.25 pts for `figsize=(12, 4)`.
- **Part b (1.0 pts):**
 - **Subplot 1 (0.5 pts):** Award 0.25 pts for `sns.barplot(data=df, x='Category', y='Revenue')`; award 0.25 pts for targeting `ax=axes[0]`.
 - **Subplot 2 (0.5 pts):** Award 0.25 pts for `sns.boxplot(data=df, x='Category', y='Revenue')`; award 0.25 pts for targeting `ax=axes[1]`.

Question 3: Layout Optimization & Chart Interpretation (0.5 pts)**Standard Model Solution**

```
# a) Layout optimization command (0.25 pts):  
plt.tight_layout()
```

Part b: Outlier Interpretation via Box Plot Whiskers (0.25 pts):

- **Model Answer:** The whiskers of a box plot typically extend up to $1.5 \times IQR$ from the quartiles ($Q_1 - 1.5 \times IQR$ and $Q_3 + 1.5 \times IQR$, where $IQR = Q_3 - Q_1$). Any transaction data points falling outside this range are plotted as distinct individual dots/markers, visually indicating they are statistical outliers.
- **Grading note:** Full credit (0.25 pts) is awarded if the student states that points beyond the whiskers (or beyond $1.5 \times IQR$) are plotted individually as outliers. Award 0.15 pts if they mention points outside the whiskers without mentioning $1.5 \times IQR$.